



universidade de aveiro

Dispositivos Conectados

Operação de um sistema embutido

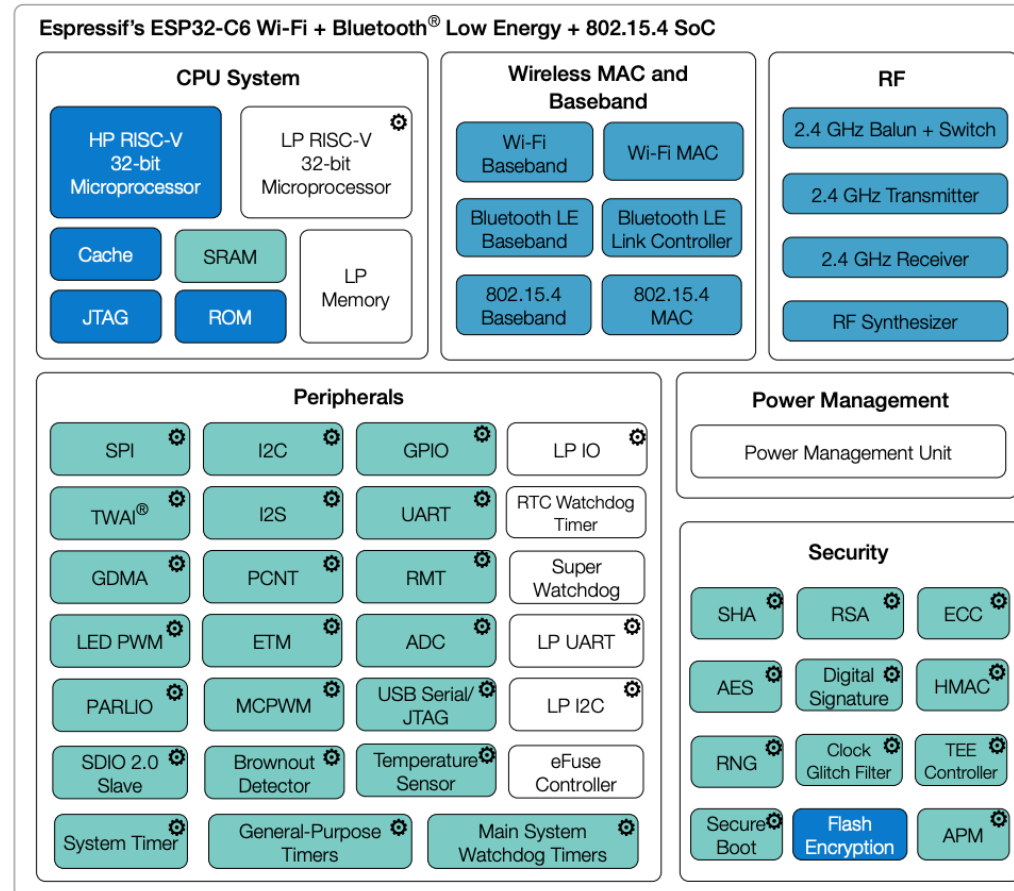
Paulo C. Bartolomeu

O que é um Sistema Embebido?

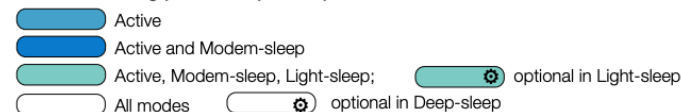
- **Combinação de *hardware* e *software*** que operam em conjunto para **realizar uma tarefa específica e pré-definida**.
- Ao contrário de um computador de uso geral (como um PC), que pode realizar uma vasta gama de tarefas, **um sistema embebido é dedicado e geralmente possui restrições de custo, energia e tamanho**.
- **Questão:** Qual é a diferença entre um *smartphone* (um computador de uso geral) e um forno micro-ondas (um sistema embebido).
 - O forno micro-ondas está permanentemente "embutido" no seu propósito.



Arquitetura do ESP32-C6



Modules having power in specific power modes:



universidade de aveiro

Arquitetura RISC-V

- Arquitetura de conjunto de instruções (ISA) aberta, o que a torna popular em sistemas embebidos.
- Integra um núcleo HP (*High-Performance*) e um núcleo LP (*Low-Power*)
- O núcleo HP pode ser dedicado à aplicação principal e o outro a tarefas de baixo nível ou de comunicação.

Multitarefa de Tempo Real

- O modelo de programação do ESP32-C6, ao usar o ESP-IDF, é um modelo de Sistema Operativo de Tempo Real (RTOS focado em Tarefas (*Tasks*) concorrentes, e não no tradicional *loop* infinito de microcontroladores simples.
 - **Multitarefa Preemptiva:** Em vez de ter um único ciclo infinito onde todas as tarefas se realizam sequencialmente, o sistema é dividido em múltiplas tarefas independentes (*tasks*). O escalonador (*scheduler*) do FreeRTOS gere estas tarefas, decidindo qual deve correr em cada momento, com base na sua prioridade e no algoritmo de escalonamento adoptado.
 - **Concorrência:** É essencial para permitir que as funcionalidades de conectividade, como as *stacks* de Wi-Fi 6, Bluetooth LE, Thread e Zigbee, corram como tarefas geridas pelo sistema, enquanto o código da aplicação executa-se noutras tarefas.
 - **A Função `app_main()`:** A função de entrada da aplicação (`app_main`) é a primeira tarefa a ser executada pelo FreeRTOS.
 - Usada para inicializar o *hardware* e o *software* e subsequentemente criar as tarefas permanentes da aplicação antes de retornar ou suspender-se.

Comunicação entre Tarefas

- **Notificações de Tarefa (Task Notifications):** Forma altamente eficiente e de baixo *overhead* para uma tarefa comunicar um evento ou transmitir dados a outra tarefa. É a primitiva de sincronização preferida do FreeRTOS em muitos casos.
- **Semáforos e Mutexes:** Utilizados para sincronização e exclusão mútua.
 -  **Mutex (Exclusive Mutual):** Usado para proteger um recurso partilhado (como um periférico ou uma variável global) de ser acedido por múltiplas tarefas ao mesmo tempo, prevenindo *race conditions*.
 -  **Semáforo (Contador ou Binário):** é usado para sinalização de eventos ou para limitar o acesso a um conjunto de recursos.
- **Filas (Queues):** Utilizadas para comunicação de dados entre tarefas.
 - Uma tarefa pode colocar dados na fila (produtor) e outra pode retirá-los (consumidor). É o principal mecanismo para desacoplar a lógica da aplicação.

Eventos e Drivers

O ESP-IDF complementa o FreeRTOS com um modelo de programação baseado em eventos para a gestão de *hardware* e comunicação:

- **Event Loop:** Os *drivers* de comunicação (como Wi-Fi e Bluetooth) funcionam com um Sistema de *Event Loop*.
 - Quando ocorre um evento de rede (ex: a conexão Wi-Fi foi estabelecida ou um pacote Bluetooth foi recebido), o sistema gera um Evento que é capturado pela aplicação.
- **Callbacks e ISRs:** O modelo faz uso intensivo de rotinas de serviço a interrupção (ISRs) de baixo nível para reagir rapidamente a eventos de *hardware* (como a mudança de estado de um pino GPIO).
 - Em geral, as ISRs fazem o mínimo de trabalho possível (apenas sinalizam uma tarefa) para depois passarem o processamento complexo para uma *Task* através de uma Fila ou Notificação.

Conceitos FreeRTOS

- **Multitarefa:** Execução de várias tarefas aparentemente em simultâneo (concorrência) através do *scheduler* do RTOS.
 - Permite que o ciclo principal da aplicação (a sua lógica) corra independentemente de tarefas de baixo nível como a gestão de rede (Wi-Fi/Bluetooth).
- **Prioridades:** Atribuição de prioridades a cada tarefa, garantindo que as funções críticas (*real-time*) são executadas primeiro.
 - Crucial para assegurar que a comunicação de rede e a resposta a eventos (ex: interrupções de sensores) ocorrem sem *delays* significativos.
- **Sincronização:** Uso de Notificações, Semáforos, Mutexes e Filas (*Queues*) para gerir o acesso a recursos partilhados (ex: uma variável global, uma porta UART, um periférico) e para comunicação entre tarefas.
 - Essencial para evitar condições de corrida (race conditions) e garantir a estabilidade do sistema.
- **Tempo Real:** Garantia de que as tarefas críticas têm um tempo de resposta previsível.
 - Permite o desenvolvimento de aplicações onde o *timing* é essencial, como controlo industrial ou processamento de áudio/vídeo.

Ambientes e linguagens de desenvolvimento p/ ESP32-C6

- **C/C++ (ESP-IDF):** Oferece o controlo máximo sobre a memória, *drivers* e o *kernel* FreeRTOS. É o ambiente recomendado para projetos robustos de IoT onde o desempenho e a gestão de recursos são críticos.
- **Python/MicroPython:** Embora o *core* do sistema ainda dependa do FreeRTOS, a utilização de linguagens de alto nível (como o MicroPython) abstrai o programador da complexidade do FreeRTOS, focando-o apenas na lógica da aplicação, à custa de alguma perda de desempenho e controlo de baixo nível.
- **Rust:** Existe também um suporte crescente para o desenvolvimento de aplicações em Rust, aproveitando as suas garantias de segurança de memória para sistemas embebidos.

FreeRTOS como base do ESP-IDF

- **Sistema Operativo:** O ESP-IDF não é apenas um conjunto de drivers e bibliotecas: é uma *framework* de desenvolvimento que é, ela própria, construída sobre o kernel FreeRTOS.
- **Ambiente de Execução:** Todas as aplicações e a maioria dos componentes *core* (como o Wi-Fi e Bluetooth) no ESP-IDF correm como Tasks do FreeRTOS.
- **Inicialização Automática:** Ao contrário de um ambiente *bare-metal* onde o programador tem de chamar `vTaskStartScheduler()`, no ESP-IDF, o FreeRTOS é iniciado automaticamente pela *framework* antes de chamar a função principal da sua aplicação (`app_main()`). A sua função `app_main` é executada como uma tarefa do FreeRTOS.

Porquê usar um sistema operativo de tempo-real?

- **Abstração das informações de tempo:** O RTOS é responsável pelo tempo de execução e fornece uma API relacionada ao tempo para a aplicação. Isso permite que a estrutura do código da aplicação seja mais direta e que o tamanho geral do código seja menor.
- **Manutenção/Extensibilidade:** abstração dos detalhes de temporização resulta em menos interdependências entre os módulos e permite que o software evolua de forma controlada e previsível. Além disso, o *kernel* é responsável pela temporização, portanto, o desempenho da aplicação é menos suscetível a alterações no *hardware* subjacente.
- **Modularidade:** As tarefas são módulos independentes, cada um dos quais deve ter um objetivo bem definido.
- **Desenvolvimento em equipa:** As tarefas também devem ter interfaces bem definidas, permitindo um desenvolvimento em equipa mais fácil.

Porquê usar um sistema operativo de tempo-real?

- **Testes mais fáceis:** Tarefas que são módulos independentes bem definidos com interfaces limpas são mais fáceis de testar isoladamente.
- **Reutilização de código:** O código projetado com maior modularidade e menos interdependências é mais fácil de reutilizar.
- **Maior eficiência:** O código da aplicação que usa um RTOS pode ser totalmente orientado a eventos. Não é necessário desperdiçar tempo de processamento a fazer *polling* a eventos que não ocorreram.
 - Opondo-se a esta eficiência, há a necessidade de processar a interrupção do RTOS e alternar a execução entre tarefas. No entanto, as aplicações que não utilizam um RTOS normalmente incluem alguma forma de interrupção de qualquer forma.
- **Tempo *Idle*:** A tarefa *idle* criada automaticamente é executada quando não há tarefas da aplicação que exijam processamento. A tarefa *idle* pode medir a capacidade de processamento disponível, realizar verificações em segundo plano ou colocar o processador em modo de baixo consumo de energia.

Porquê usar um sistema operativo de tempo-real?

- **Gestão de energia:** Os ganhos de eficiência resultantes da utilização de um RTOS permitem que o processador passe mais tempo em modo de baixo consumo de energia. O consumo de energia pode ser reduzido significativamente colocando o processador num estado de baixo consumo cada vez que a tarefa *idle* é executada.
- **Tratamento flexível de interrupções:** Os manipuladores (*handlers*) de interrupções podem ser mantidos muito curtos, adiando o processamento para uma tarefa criada pelo programador da aplicação ou para a tarefa *daemon* RTOS criada automaticamente (também conhecida como *timer task*).
- **Requisitos de processamento mistos:** Pode-se alcançar uma mistura de processamento periódico, contínuo e orientado a eventos dentro de uma aplicação. Além disso, os requisitos de tempo real rígidos e flexíveis podem ser atendidos selecionando as prioridades apropriadas de tarefas e interrupções.



Kahoot!

Time



universidade de aveiro



theoria poiesis praxis